

1. Screenshots que comprovem as tabelas carregadas no Postgres.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'public' schema and the 'dadostesouroipca' table. The main pane shows the query result for the following SQL query:

```
SELECT "CompraManha", "VendaManha", "PUCompraManha", "PUVendaManha", "PUBaseManha", "Data_Vencimento"
FROM public.dadostesouroipca;
```

The query result is displayed in a table with 15 rows and 8 columns. The columns are: CompraManha (double precision), VendaManha (double precision), PUCompraManha (double precision), PUVendaManha (double precision), PUGBaseManha (double precision), Data_Vencimento (timestamp without time zone), Data_Base (timestamp without time zone), and Tipo (text).

	CompraManha	VendaManha	PUCompraManha	PUVendaManha	PUGBaseManha	Data_Vencimento	Data_Base	Tipo
1	9.03	9.13	1025.39	1014.49	1013.89	2045-05-15 00:00:00	2005-01-11 00:00:00	Tesouro IPCA+ com Juros S
2	8.8	8.88	1229.35	1222.62	1221.92	2015-05-15 00:00:00	2005-01-11 00:00:00	Tesouro IPCA+ com Juros S
3	8.82	8.9	1144.38	1135.84	1135.18	2024-08-15 00:00:00	2005-01-11 00:00:00	Tesouro IPCA+ com Juros S
4	8.83	8.87	1355.58	1353.69	1352.9	2009-05-15 00:00:00	2005-01-11 00:00:00	Tesouro IPCA+ com Juros S
5	8.8	8.82	1461.24	1460.84	1460	2006-08-15 00:00:00	2005-01-11 00:00:00	Tesouro IPCA+ com Juros S
6	8.78	8.8	1460.81	1460.4	1459.56	2006-08-15 00:00:00	2005-01-11 00:00:00	Tesouro IPCA+ com Juros S
7	8.83	8.87	1354.8	1352.9	1352.12	2009-05-15 00:00:00	2005-01-10 00:00:00	Tesouro IPCA+ com Juros S
8	9.03	9.13	1024.79	1013.89	1013.29	2045-05-15 00:00:00	2005-01-10 00:00:00	Tesouro IPCA+ com Juros S
9	8.82	8.9	1143.72	1135.18	1134.52	2024-08-15 00:00:00	2005-01-10 00:00:00	Tesouro IPCA+ com Juros S
10	8.79	8.87	1229.48	1222.76	1222.05	2015-05-15 00:00:00	2005-01-10 00:00:00	Tesouro IPCA+ com Juros S
11	8.83	8.91	1141.99	1133.46	1132.26	2024-08-15 00:00:00	2005-01-07 00:00:00	Tesouro IPCA+ com Juros S
12	9.04	9.14	1023.09	1011.22	1011.13	2045-05-15 00:00:00	2005-01-07 00:00:00	Tesouro IPCA+ com Juros S
13	8.78	8.86	1229.62	1222.89	1221.59	2015-05-15 00:00:00	2005-01-07 00:00:00	Tesouro IPCA+ com Juros S
14	8.81	8.85	1354.96	1353.07	1351.63	2009-05-15 00:00:00	2005-01-07 00:00:00	Tesouro IPCA+ com Juros S
15	8.75	8.77	1460.57	1460.17	1458.63	2006-08-15 00:00:00	2005-01-07 00:00:00	Tesouro IPCA+ com Juros S

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'public' schema and the 'dadostesouropre' table. The main pane shows the query result for the following SQL query:

```
SELECT "CompraManha", "VendaManha", "PUCompraManha", "PUVendaManha", "PUBaseManha", "Data_Vencimento"
FROM public.dadostesouropre;
```

The query result is displayed in a table with 17 rows and 8 columns. The columns are: CompraManha (double precision), VendaManha (double precision), PUCompraManha (double precision), PUVendaManha (double precision), PUGBaseManha (double precision), Data_Vencimento (timestamp without time zone), Data_Base (timestamp without time zone), and Tipo (text).

	CompraManha	VendaManha	PUCompraManha	PUVendaManha	PUGBaseManha	Data_Vencimento	Data_Base	Tipo
1	18.74	18.78	923.34	923.2	922.57	2005-07-01 00:00:00	2005-01-11 00:00:00	Tesouro Prefixado
2	18.44	18.48	964.38	964.31	963.66	2005-04-01 00:00:00	2005-01-11 00:00:00	Tesouro Prefixado
3	18.69	18.75	883.6	883.27	882.67	2005-10-01 00:00:00	2005-01-11 00:00:00	Tesouro Prefixado
4	17.54	17.64	845.27	843.41	842.87	2008-01-01 00:00:00	2005-01-11 00:00:00	Tesouro Prefixado com
5	18.44	18.48	963.73	963.66	963.01	2005-04-01 00:00:00	2005-01-10 00:00:00	Tesouro Prefixado
6	18.7	18.74	922.86	922.71	922.09	2005-07-01 00:00:00	2005-01-10 00:00:00	Tesouro Prefixado
7	18.67	18.73	883.1	882.78	882.18	2005-10-01 00:00:00	2005-01-10 00:00:00	Tesouro Prefixado
8	18.34	18.4	848.98	848.56	847.99	2006-01-01 00:00:00	2005-01-10 00:00:00	Tesouro Prefixado
9	17.89	17.97	785.84	785.06	784.55	2006-07-01 00:00:00	2005-01-10 00:00:00	Tesouro Prefixado
10	17.53	17.63	844.91	843.05	842.51	2008-01-01 00:00:00	2005-01-10 00:00:00	Tesouro Prefixado com
11	18.47	18.51	963.03	962.96	962.31	2005-04-01 00:00:00	2005-01-07 00:00:00	Tesouro Prefixado
12	18.73	18.77	922.12	921.98	921.35	2005-07-01 00:00:00	2005-01-07 00:00:00	Tesouro Prefixado
13	18.71	18.77	882.29	881.96	881.36	2005-10-01 00:00:00	2005-01-07 00:00:00	Tesouro Prefixado
14	17.98	18.06	784.45	783.67	783.15	2006-07-01 00:00:00	2005-01-07 00:00:00	Tesouro Prefixado
15	18.43	18.49	847.78	847.36	846.79	2006-01-01 00:00:00	2005-01-07 00:00:00	Tesouro Prefixado
16	17.59	17.69	843.25	841.4	840.85	2008-01-01 00:00:00	2005-01-07 00:00:00	Tesouro Prefixado com
17	18.42	18.46	962.48	962.4	961.76	2005-04-01 00:00:00	2005-01-06 00:00:00	Tesouro Prefixado

Total rows: 54620 Query complete 00:00:00.184 CRLF Ln 1, Co

2. Screenshots ou logs que comprovem os Código Spark para os pipelines (incluindo consultas Spark SQL)

ETL Spark — Bronze → Silver → Gold (Tesouro Direto)

Lê os dados brutos (JSON) entregues pelo Kafka Connect no **S3** (Bronze) e gera **Silver** (descarte de inválidos, dedup por chave de negócio, datas, normalização) e **Gold** (agregações por título), gravando Parquet de volta no S3.

```
[9]: import sys; print(sys.executable)
/opt/conda/bin/python

[10]: import os
os.environ["PYSPARK_SUBMIT_ARGS"] = (
    "--packages org.apache.hadoop:hadoop-aws:3.3.4,"
    "com.amazonaws:aws-java-sdk-bundle:1.12.262 pyspark-shell"
)

[11]: from pyspark.sql import SparkSession, Window
from pyspark.sql.functions import (col, from_unixtime, to_date, avg, count,
    round as round, row_number, min as min, max as max)

spark = (SparkSession.builder.appName("ETL Tesouro - Bronze/Silver/Gold")
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
        "com.amazonaws.auth.EnvironmentVariableCredentialsProvider")
    .getOrCreate())
spark.sparkContext.setLogLevel("WARN")
CHAVE = ["Tipo", "Data_Vencimento", "Data_Base"] # chave de negócio
```

Bronze — leitura dos JSON brutos no S3

```
[12]: BUCKET_IPCA = "s3a://my-bucket-cabovier-01"
BUCKET_PRE = "s3a://my-bucket-cabovier-02"
df_bronze_ipca = spark.read.json(f"{BUCKET_IPCA}/raw-data/kafka/postgres-dadostesouroipca/")
df_bronze_pre = spark.read.json(f"{BUCKET_PRE}/raw-data/kafka/postgres-dadostesouropre/")
print("Bronze IPCA:", df_bronze_ipca.count(), ", Bronze PRE:", df_bronze_pre.count())
df_bronze_ipca.printSchema(); df_bronze_ipca.show(5, truncate=False)

Bronze IPCA: 500 | Bronze PRE: 500
root
 |-- CompraAnha: double (nullable = true)
 |-- Data_Base: long (nullable = true)
 |-- Data_Vencimento: long (nullable = true)
 |-- PUBaseAnha: double (nullable = true)
 |-- PUCompraAnha: double (nullable = true)
 |-- PUVendaAnha: double (nullable = true)
 |-- Tipo: string (nullable = true)
 |-- VendaAnha: double (nullable = true)
 |-- dt_update: long (nullable = true)
 |-- partition: integer (nullable = true)

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|CompraAnha|Data_Base|Data_Vencimento|PUBaseAnha|PUCompraAnha|PUVendaAnha|Tipo|VendaAnha|dt_update|partition|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|10.04|1144281600000|1179187200000|1567.32|1568.45|1568.15|Tesouro IPCA+ com Juros Semestrais|10.06|1781269498097|0|
|10.09|1144281600000|1155600000000|1580.63|1581.61|1581.51|Tesouro IPCA+ com Juros Semestrais|10.71|1781269498097|0|
|11.37|1141344000000|1155600000000|1553.25|1554.75|1554.63|Tesouro IPCA+ com Juros Semestrais|11.39|1781269498097|0|
|10.31|1141344000000|1179187200000|1548.96|1542.58|1542.27|Tesouro IPCA+ com Juros Semestrais|10.33|1781269498097|0|
|10.07|1144108800000|1179187200000|1565.19|1566.33|1566.03|Tesouro IPCA+ com Juros Semestrais|10.09|1781269498097|0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Silver — qualidade e transformação

- Descarta registros inválidos (PU de compra/venda ausente).
- Deduplica por chave de negócio (Tipo+Vencimento+Base), mantendo o `dt_update` mais recente.
- Converte timestamps (epoch ms → data) e normaliza.

```
[13]: def to_silver(df):
    df_valid = df.filter(col("PUCompraAnha").isNotNull() & col("PUVendaAnha").isNotNull())
    w = Window.partitionBy("CHAVE").orderBy(col("dt_update").desc())
    df_dedup = (df_valid.withColumn("rn", row_number().over(w))
        .filter(col("rn") == 1).drop("rn"))

    return df_dedup
        .withColumn("Data_Vencimento", to_date(from_unixtime(col("Data_Vencimento")/1000)))
        .withColumn("Data_Base", to_date(from_unixtime(col("Data_Base")/1000)))
        .withColumn("dt_update", from_unixtime(col("dt_update")/1000, "yyyy-MM-dd HH:mm:ss"))

df_silver_ipca = to_silver(df_bronze_ipca); df_silver_pre = to_silver(df_bronze_pre)
print("Silver IPCA:", df_silver_ipca.count(), ", Silver PRE:", df_silver_pre.count())
df_silver_ipca.orderBy("Data_Base").show(5, truncate=False)
df_silver_ipca.write.mode("overwrite").parquet(f"{BUCKET_IPCA}/processed-data/ipca/silver/")
df_silver_pre.write.mode("overwrite").parquet(f"{BUCKET_PRE}/processed-data/pre/silver/")

Silver IPCA: 500 | Silver PRE: 500

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|CompraAnha|Data_Base|Data_Vencimento|PUBaseAnha|PUCompraAnha|PUVendaAnha|Tipo|VendaAnha|dt_update|partition|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|8.78|2004-12-31|2009-05-15|1348.51|1351.84|1349.94|Tesouro IPCA+ com Juros Semestrais|8.02|2026-06-12 13:04:58|0|
|8.76|2004-12-31|2015-05-15|1219.16|1227.19|1220.45|Tesouro IPCA+ com Juros Semestrais|8.04|2026-06-12 13:04:58|0|
|8.71|2004-12-31|2006-08-15|1454.56|1456.5|1456.09|Tesouro IPCA+ com Juros Semestrais|8.73|2026-06-12 13:04:58|0|
|8.83|2004-12-31|2024-08-15|1128.44|1138.15|1129.64|Tesouro IPCA+ com Juros Semestrais|8.91|2026-06-12 13:04:58|0|
|9.05|2004-12-31|2045-05-15|1006.61|1018.52|1007.68|Tesouro IPCA+ com Juros Semestrais|9.15|2026-06-12 13:04:58|0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Gold — agregações analíticas por título

```
[14]: def to_gold(df):
      return (df.groupby("Tipo").agg(
          sround(avg("PUCompranhanha"),2).alias("Media_PUCompranhanha"),
          sround(avg("PUVendahanha"),2).alias("Media_PUVendahanha"),
          sround(avg("Compranhanha"),4).alias("Media_TaxaCompra"),
          sround(smin("PUCompranhanha"),2).alias("Min_PUCompra"),
          sround(smax("PUCompranhanha"),2).alias("Max_PUCompra"),
          count("").alias("Total_Registros")))
      df_gold_ipca = to_gold(df_silver_ipca); df_gold_pre = to_gold(df_silver_pre)
      df_gold_ipca.show(truncate=False); df_gold_pre.show(truncate=False)
      df_gold_ipca.write.mode("overwrite").parquet(f"{BUCKET_IPCA}/analytics/ipca/gold/")
      df_gold_pre.write.mode("overwrite").parquet(f"{BUCKET_PRE}/analytics/pre/gold/")
```

Tipo	Media_PUCompranhanha	Media_PUVendahanha	Media_TaxaCompra	Min_PUCompra	Max_PUCompra	Total_Registros
Tesouro IPCA+	612.6	607.7	7.6588	484.02	835.99	66
Tesouro IPCA+ com Juros Semestrais	1383.67	1378.66	9.085	1018.52	1592.49	434

Tipo	Media_PUCompranhanha	Media_PUVendahanha	Media_TaxaCompra	Min_PUCompra	Max_PUCompra	Total_Registros
Tesouro Prefixado com Juros Semestrais	892.45	890.99	15.2922	840.92	966.66	115
Tesouro Prefixado	872.22	871.88	16.2918	728.38	1000.0	385

Validação (Spark SQL)

```
[15]: df_gold_ipca.unionByName(df_gold_pre).createOrReplaceTempView("gold_tesouro")
      spark.sql("SELECT Tipo, Total_Registros, Media_PUCompranhanha FROM gold_tesouro "
      "ORDER BY Media_PUCompranhanha DESC").show(truncate=False)
```

Tipo	Total_Registros	Media_PUCompranhanha
Tesouro IPCA+ com Juros Semestrais	434	1383.67
Tesouro Prefixado com Juros Semestrais	115	892.45
Tesouro Prefixado	385	872.22
Tesouro IPCA+	66	612.6

3. Screenshots ou logs que comprovem os Dados processados no Amazon S3, organizados e particionados.

Buckets

Buckets de uso geral

Todas as regiões da AWS

Buckets de diretórios

Buckets de uso geral (2) Informações



Copiar ARN

Vazio

Excluir

Criar bucket

Buckets são contêineres para dados armazenados no S3.

Q Encontrar buckets por nome

< 1 > ⚙

Nome	Região da AWS	Data de criação
my-bucket-cabxavier-01	Leste dos EUA (Norte da Virgínia) us-east-1	11 Jun 2026 03:36:42 PM -03
my-bucket-cabxavier-02	Leste dos EUA (Norte da Virgínia) us-east-1	11 Jun 2026 03:37:26 PM -03

my-bucket-cabxavier-01 Informações

Objetos

Metadados

Propriedades

Permissões

Métricas

Gerenciamento

Sistemas de arquivos (novo)

Pontos de acesso

Objetos (3)



Copiar URI do S3

Copiar URL

Fazer download

Abrir ↗

Excluir

Ações ▼

Criar pasta

Carregar ↗

Os objetos são as entidades fundamentais armazenadas no Amazon S3. Você pode usar o [inventário do Amazon S3](#) para obter uma lista de todos os objetos em seu bucket. Para outras pessoas acessarem seus objetos, você precisará conceder permissões explicitamente a eles. [Saiba mais](#)

Q Localizar objetos por prefixo

< 1 > ⚙

Nome	Tipo	Última modificação	Tamanho	Classe de armazenamento
analytics/	Pasta	-	-	-
processed-data/	Pasta	-	-	-
raw-data/	Pasta	-	-	-

gold/

Copiar URI do S3

Objetos

Propriedades

Objetos (2)

Copiar URI do S3

Copiar URL

Fazer download

Abrir L2

Excluir

Ações

Criar pasta

Carregar

Os objetos são as entidades fundamentais armazenadas no Amazon S3. Você pode usar o [inventário do Amazon S3](#) para obter uma lista de todos os objetos em seu bucket. Para outras pessoas acessarem seus objetos, você precisará conceder permissões explicitamente a eles. [Saiba mais](#)

Localizar objetos por prefixo

< 1 >

☐	Nome	Tipo	Última modificação	Tamanho	Classe de armazenamento
☐	_SUCCESS	-	12 Jun 2026 04:12:23 PM -03	0 B	Padrão
☐	part-00000-db4eb006-6825-4068-b510-e06fdb03b5a-c000.snappy.parquet	parquet	12 Jun 2026 04:12:19 PM -03	2.3 KB	Padrão

silver/

Copiar URI do S3

Objetos

Propriedades

Objetos (2)

Copiar URI do S3

Copiar URL

Fazer download

Abrir L2

Excluir

Ações

Criar pasta

Carregar

Os objetos são as entidades fundamentais armazenadas no Amazon S3. Você pode usar o [inventário do Amazon S3](#) para obter uma lista de todos os objetos em seu bucket. Para outras pessoas acessarem seus objetos, você precisará conceder permissões explicitamente a eles. [Saiba mais](#)

Localizar objetos por prefixo

< 1 >

☐	Nome	Tipo	Última modificação	Tamanho	Classe de armazenamento
☐	_SUCCESS	-	12 Jun 2026 04:11:20 PM -03	0 B	Padrão
☐	part-00000-fcb1651f-3979-4744-92cb-ee800d1e2780-c000.snappy.parquet	parquet	12 Jun 2026 04:11:17 PM -03	13.4 KB	Padrão

partition=0/

Copiar URI do S3

Objetos

Propriedades

Objetos (500)

Copiar URI do S3

Copiar URL

Fazer download

Abrir L2

Excluir

Ações

Criar pasta

Carregar

Os objetos são as entidades fundamentais armazenadas no Amazon S3. Você pode usar o [inventário do Amazon S3](#) para obter uma lista de todos os objetos em seu bucket. Para outras pessoas acessarem seus objetos, você precisará conceder permissões explicitamente a eles. [Saiba mais](#)

Localizar objetos por prefixo

< 1 2 >

☐	Nome	Tipo	Última modificação	Tamanho	Classe de armazenamento
☐	postgres-dadostesouroipca+0+00000000000.json	json	12 Jun 2026 04:07:20 PM -03	471.0 B	Padrão
☐	postgres-dadostesouroipca+0+00000000002.json	json	12 Jun 2026 04:07:21 PM -03	470.0 B	Padrão
☐	postgres-dadostesouroipca+0+00000000004.json	json	12 Jun 2026 04:07:22 PM -03	468.0 B	Padrão

my-bucket-cabxavier-02 Informações

Objetos

Metadados

Propriedades

Permissões

Métricas

Gerenciamento

Sistemas de arquivos (novo)

Pontos de acesso

Objetos (3)

Copiar URI do S3

Copiar URL

Fazer download

Abrir

Excluir

Ações

Criar pasta

Carregar

Os objetos são as entidades fundamentais armazenadas no Amazon S3. Você pode usar o [inventário do Amazon S3](#) para obter uma lista de todos os objetos em seu bucket. Para outras pessoas acessarem seus objetos, você precisará conceder permissões explicitamente a eles. [Saiba mais](#)

Localizar objetos por prefixo

< 1 >

☐	Nome	Tipo	Última modificação	Tamanho	Classe de armazenamento
☐	analytics/	Pasta	-	-	-
☐	processed-data/	Pasta	-	-	-
☐	raw-data/	Pasta	-	-	-

gold/

Copiar URI do S3

Objetos

Propriedades

Objetos (2)

Copiar URI do S3

Copiar URL

Fazer download

Abrir

Excluir

Ações

Criar pasta

Carregar

Os objetos são as entidades fundamentais armazenadas no Amazon S3. Você pode usar o [inventário do Amazon S3](#) para obter uma lista de todos os objetos em seu bucket. Para outras pessoas acessarem seus objetos, você precisará conceder permissões explicitamente a eles. [Saiba mais](#)

Localizar objetos por prefixo

< 1 >

☐	Nome	Tipo	Última modificação	Tamanho	Classe de armazenamento
☐	_SUCCESS	-	12 Jun 2026 04:12:47 PM -03	0 B	Padrão
☐	part-00000-a3876d80-8f54-4c4e-b867-650e4c91d50b-c000.snappy.parquet	parquet	12 Jun 2026 04:12:44 PM -03	2.3 KB	Padrão

silver/

Copiar URI do S3

Objetos

Propriedades

Objetos (2)

Copiar URI do S3

Copiar URL

Fazer download

Abrir

Excluir

Ações

Criar pasta

Carregar

Os objetos são as entidades fundamentais armazenadas no Amazon S3. Você pode usar o [inventário do Amazon S3](#) para obter uma lista de todos os objetos em seu bucket. Para outras pessoas acessarem seus objetos, você precisará conceder permissões explicitamente a eles. [Saiba mais](#)

Localizar objetos por prefixo

< 1 >

☐	Nome	Tipo	Última modificação	Tamanho	Classe de armazenamento
☐	_SUCCESS	-	12 Jun 2026 04:11:43 PM -03	0 B	Padrão
☐	part-00000-050a8b9e-d110-4548-aff9-f99febf4f646-c000.snappy.parquet	parquet	12 Jun 2026 04:11:39 PM -03	13.7 KB	Padrão

Objetos (500)

Copiar URI do S3 Copiar URL Fazer download Abrir Excluir Ações Criar pasta Carregar

Os objetos são as entidades fundamentais armazenadas no Amazon S3. Você pode usar o inventário do Amazon S3 para obter uma lista de todos os objetos em seu bucket. Para outras pessoas acessarem seus objetos, você precisará conceder permissões explicitamente a eles. Saiba mais

Localizar objetos por prefixo

1 2

☐	Nome	Tipo	Última modificação	Tamanho	Classe de armazenamento
☐	postgres-dadosesoupre+0+0000000000.json	json	12 Jun 2026 04:07:21 PM -03	435.0 B	Padrão
☐	postgres-dadosesoupre+0+0000000002.json	json	12 Jun 2026 04:07:22 PM -03	456.0 B	Padrão
☐	postgres-dadosesoupre+0+0000000004.json	json	12 Jun 2026 04:07:23 PM -03	435.0 B	Padrão